

DYNAMIC PROGRAMMING ↗ optimal subproblem
↘ overlapping substructures

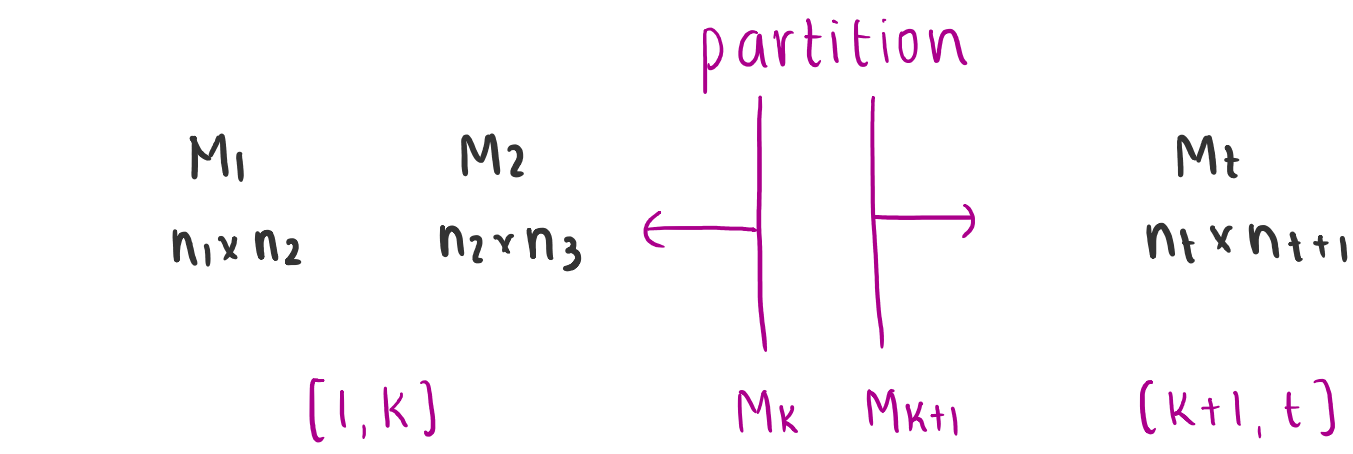
• Matrix Chain Multiplication

$M_1 M_2 \dots M_t$
 M_1, M_2
 $k \times l \quad l \times s$
 $O(kls)$
If each M_i is $n \times n$,
 $O(t \cdot n^3)$
of mults ↗ each mult.

Say each M_i is of dimension $n_i \times n_{i+1}$.

e.g. $A \quad B \quad C$
 $6 \times 2 \quad 2 \times 4 \quad 4 \times 3$

$(AB)C$
 $(6 \times 2 \times 4) + (6 \times 4 \times 3)$
120
 $A(BC)$
 $(6 \times 2 \times 3) + (2 \times 4 \times 3)$
60

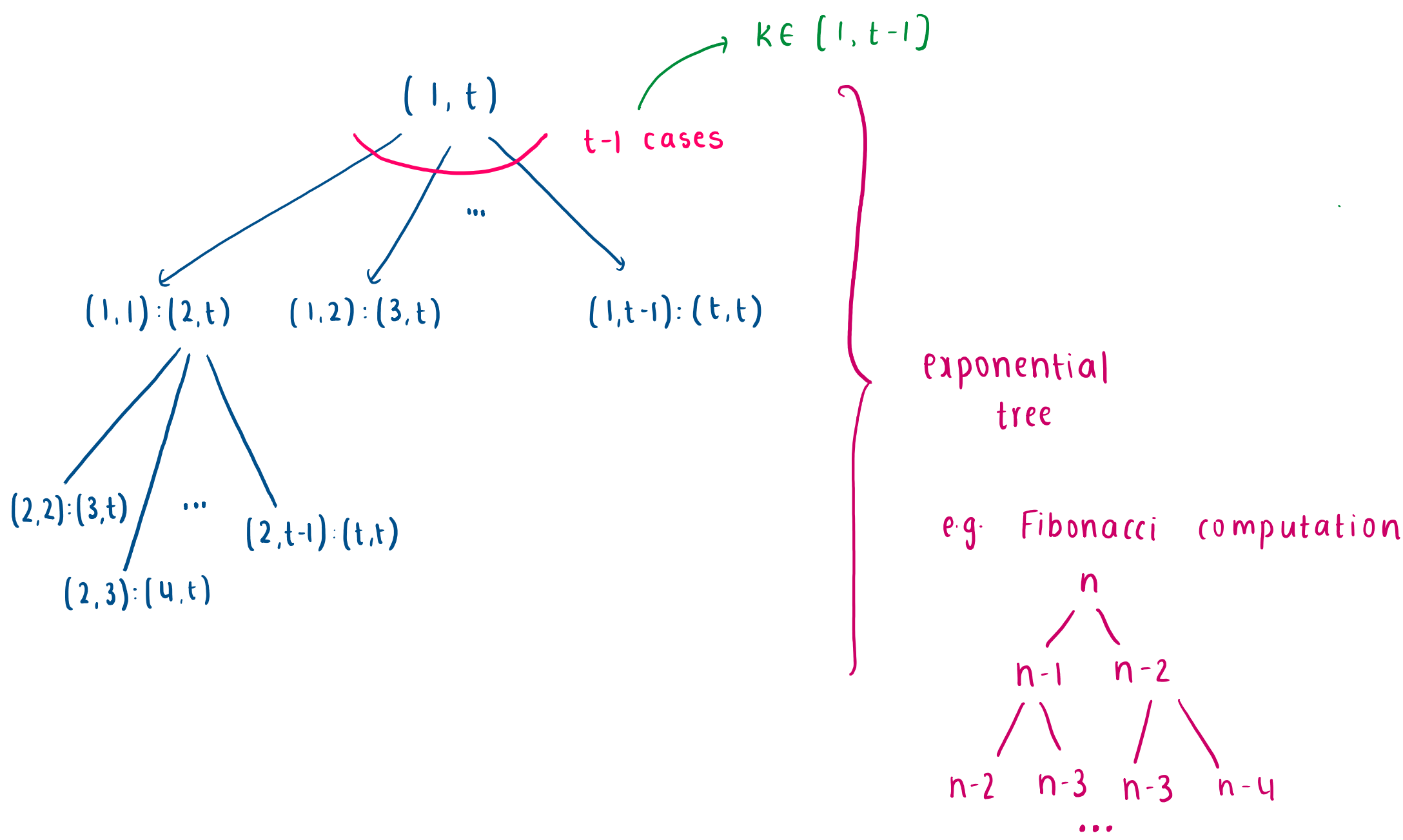


$MCM(1, t) \xrightarrow{k \in [1, t-1]} MCM(1, k) \cdot \text{merge} \cdot MCM(k+1, t)$
↗ matrix chain multiplication
↖ matrix chain product
 $T(1, k) + T(k+1, t) + n_1 \times n_{k+1} \times n_{t+1}$

$T(1, t) = \min_{k \in [1, t-1]} \{ T(1, k) + T(k+1, t) + n_1 \times n_{k+1} \times n_{t+1} \}$

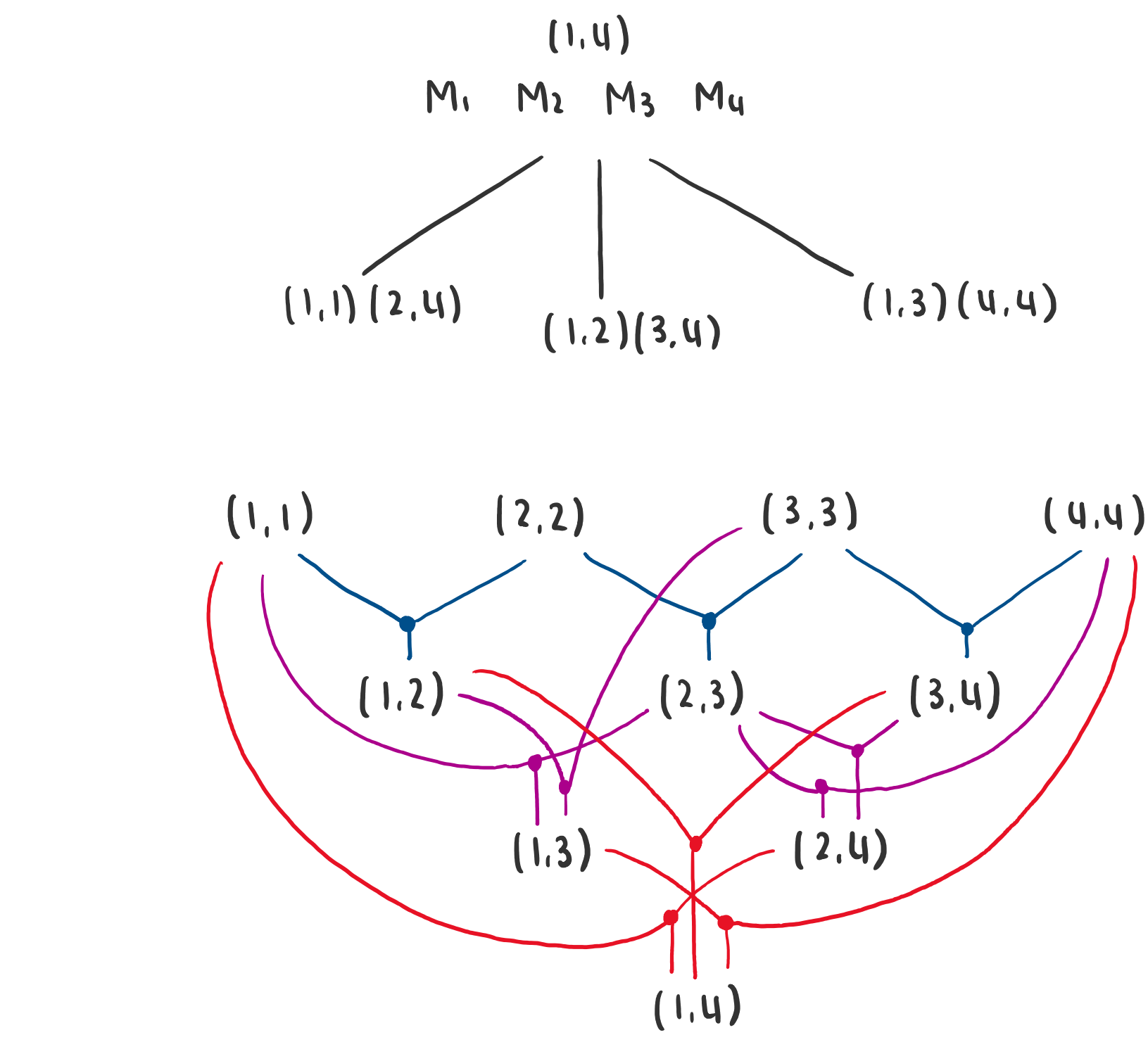
e.g. $A \quad B \quad C$
 $1 \quad 2 \quad 3$
 $n_1 = 6, n_2 = 2, n_3 = 4, n_4 = 3$

$T(1, 3) = \min \{ T(1, 1) + T(2, 3) + n_1 \times n_2 \times n_4 ; T(1, 2) + T(3, 3) + n_1 \times n_3 \times n_4 \}$

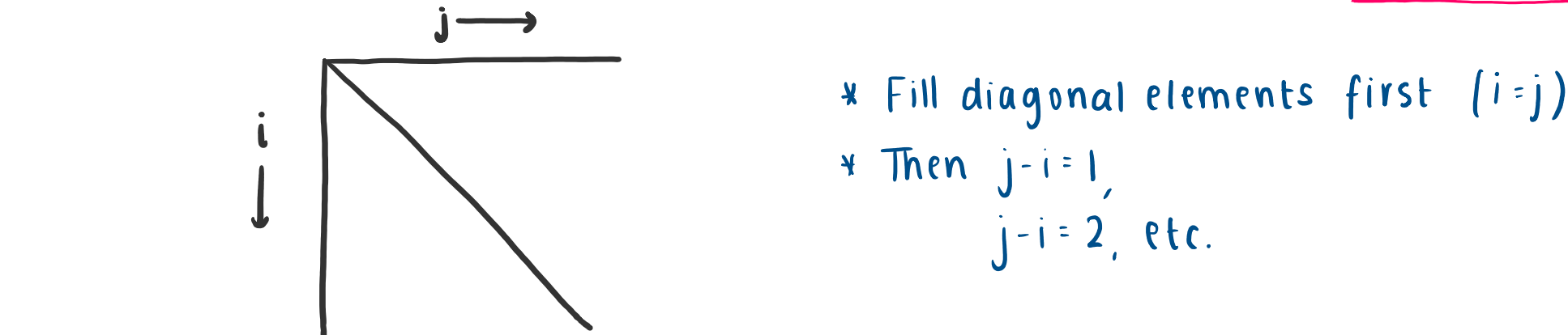


∴ Better to store computations and reuse
"Make good use of space to save time"

$T(i, j)$ $i \leq j$
of such cases: $\binom{n}{2} + n$
not n^2 since $i \leq j$
↗ $i \neq j, i \leq j$ ↖ $i = j$



"We are building a lookup table for computations for efficient reuse"
MEMORIZATION



We begin by calculating $T(i, i)$ for all i ,
then $T(i, i+1)$ for all i , next $T(i, i+2)$ and so on.

In this way, the terms $T(i, k)$ and $T(k+1, j)$ will be
available when we calculate $T(i, j)$.

$T(i, j) = \min_{k \in [i, j-1]} \{ T(i, k) + T(k+1, j) + n_i \times n_{k+1} \times n_{j+1} \}$

L operations, each involves 2 lookups and 1 matrix mult.

$2L + L \cdot n^3$
↗ $n_x \times n_y \times n_z$
↖ not really n^3 since we're just doing integer mult, not actually multiplying matrices.

$\left(\binom{t}{2} + t\right)$ entries in lookup table
↗ $\leq 2(j-1)$

For each entry, we make at most $2t$ lookups
and t arithmetic computations
↖ $\leq (j-1)$
since $k \in [i, j-1]$